Bucketization: Comparison-Free Re-Sorting of Multi-Column Views

Attila Torda

October 2026

Abstract

A file manager, a spreadsheet or an asset browser shows a list that the user can order by any of several columns. Each click on a column header usually re-sorts the whole list with a comparison sort, often under locale-aware string collation, before the first row can be drawn. On folders of 100,000 entries this takes hundreds of milliseconds to seconds in the file managers we measured. We describe *Bucketization*, a simple combination of known ideas that removes comparisons from the click. When an element is loaded, it is assigned once, for every column, to one of at most a few dozen buckets whose boundaries come from a sorted sample and whose sizes are planned: the buckets at both ends hold one screen of rows and the sizes grow geometrically towards the middle. A click then regroups the elements by their stored bucket ids with one counting pass and sorts only the bucket on screen; the remaining buckets are sorted when they scroll into view. The final order is exact, because buckets decide only *when* an element is sorted, never *where* it ends up. We measured Bucketization inside the sort code of ten Linux file managers, the KDE file dialogs and the Godot editor, on real directory listings of 36,000–100,000 entries. The time until the visible rows are final drops from 4.3–3,050 ms to 0.047–3.0 ms (median speed-up 826×), for a one-time pre-sort of a median 7.0 ms per column, and every one of the 510 orders checked matches the project's own comparison. A second machine, under Windows, confirms the gains. We also report where Bucketization does not help, three defects in the projects' comparators that the measurements exposed, and a violation of the C standard's `strxfrm`/`strcoll` contract in the Windows C runtime that makes one column's key unusable there.

1 Introduction

Sorting a list by a column is among the most common operations of desktop software. File managers order a directory by name, size, type or modification time; email clients by sender or date; spreadsheets and database front ends by any column. Users switch between these orders often, and they expect the view to follow at once.

Most implementations re-sort the whole collection on every switch. For n elements this costs $\Theta(n \log n)$ comparisons before the first row can be shown, and in user-facing software a comparison is rarely cheap: file names are compared under the user's locale (Unicode collation [2]), often with "natural" handling of embedded numbers, and with fallbacks such as "folders first" and "then by name". In KDE Dolphin, for example, sorting 98,000 file names takes 550 ms on our reference machine (Section 6); the bulk of it is `QCollator` comparisons.

The user, however, only sees one screen of rows. Partial sorting algorithms [6, 7] exploit this: they produce the first p elements of the order in $O(n + p \log p)$ time. But they still compare every element on every switch, they repeat their work when the user switches back, and they degrade when many keys are equal.

Bucketization moves the comparisons from the click to the moment an element is loaded, and pays for them once per column instead of once per click:

1. for every column (*sort key*), each element is assigned to a bucket by a binary search over a few split points taken from a sorted sample, and the bucket id is stored in two bytes;
2. a click regroups the elements by bucket id with a counting sort, which needs no comparisons, and the grouping is cached per column;
3. a bucket is sorted the first time a row inside it is read. The buckets at both ends of the order are planned to hold about one screen of rows, so the first screen in either direction costs $O(p \log p)$ comparisons.

None of these steps is new on its own: assigning elements to buckets through sampled splitters is the core of samplesort [3, 8], deferring sorting work until a query needs it is the idea behind database cracking [5] and incremental sorting [7], and order-preserving fixed-size key prefixes are the "poor man's normalized keys" of database sorting [4] and PostgreSQL's abbreviated keys [10]. Our contribution is the combination, tuned for interactive multi-column views, and its evaluation in real software:

- a design with *planned, unequal* bucket sizes, stored per-key bucket ids, cached groupings and lazy per-bucket sorting, which makes a column switch comparison-free and keeps the order exact (Sections 3 and 4);
- handling of what real file managers need: descending order that keeps "folders first", runs of equal keys, comparators without a key, and comparators that are not even consistent;
- an open-source C++17 implementation with a Java port and a cross-language specification (Section 5);
- measurements inside the actual sort paths of twelve projects on real listings, including the cases where Bucketization does not help (Section 6);
- three defects in those projects' comparators and one in the Windows C runtime's collation functions, found because Bucketization's results are checked against every project's own order (Sections 6.5 and 7).

2 Problem

A *view* holds a collection S of n elements and a set of m sort keys. Sort key j is a total preorder $\preceq_j$ on elements, usually given as a comparator, possibly combined with a tie-breaker; elements that compare equal keep their insertion order. The view shows a window of p consecutive positions (one screen; $p = 64$ in our experiments) of the current order, which is sort key j ascending or descending. The operations are:

- Add, Remove, Update of elements, mostly in bulk when a directory is listed;
- SortBy(j, *dir*), the column click;
- Read($i, i + p$): the elements at positions $i, \ldots, i + p - 1$ of the current order, in that order.

The latency the user perceives after a click is that of SortBy followed by Read of the visible window, usually the first one. We want it small even when comparisons are expensive, and we want the result to be exactly the order a full stable sort would produce.

3 Design

3.1 Sort keys and normalized keys

A sort key is either *keyed* or *comparator-only*. A keyed sort key maps every element x to a 64-bit unsigned *normalized key* $\kappa(x)$ whose numeric order agrees with the element order: $\kappa(x) < \kappa(y)$ implies $x \prec y$. Its order is defined as *normalized key first, then the tie-breaker*, so the two can never contradict each other. Integers map exactly (signed values with the sign bit flipped), floating-point values by the IEEE 754 total order, and strings by their first eight bytes, big-endian, or the first eight bytes of a collation sort key such as ICU's or the output of `strxfrm`. String keys are prefix keys: names that share their first eight bytes get the same normalized key and are ordered by the tie-breaker.

A comparator-only sort key is just a comparator, for orders without a simple prefix key, such as natural ordering of embedded numbers (`file2` before `file10`).

3.2 Bucket plans

A *bucket plan* sets the target size of every bucket. The default plan, *geometric*, gives the first and the last bucket the size of one page p and doubles the sizes towards the middle, up to a cap of 64 buckets; for $n = 1000$ and $p = 50$ the targets are $[50, 100, 200, 300, 200, 100, 50]$. The ends are small because a view shows the start of the order first, and the end first after a click on the same column (descending). Sizes grow towards the middle so that the number of buckets, and with it the cost of assigning an element, stays logarithmic in n/p. Other plans (equal sizes, explicit sizes, one page at the front only) are available; the plan affects only the cost of reads, never the order.

3.3 Layout estimation

For each sort key, a *layout* turns the plan into split points. The implementation draws a jittered systematic sample of $s = 2048$ elements (one random element from each of s equal strides of the collection), sorts it, and places split b at sample position $\lfloor (t_1 + \cdots + t_b)\, s/n \rfloor$, where t_i are the target sizes. Bucket b then holds the elements x with $\sigma_{b-1} \preceq x \prec \sigma_b$, where σ_b is the b-th split point. Equal elements must share a bucket, so a split that falls inside a run of equal sample values moves to the nearer end of the run, and splits that coincide are merged.

For keyed sort keys, split points are normalized keys, and an element's bucket is an upper-bound search over at most 63 integers. When many elements share a normalized key (camera files named `IMG_2024...`, or a column where every value is equal), moving the split to the end of the run would put the whole run into one bucket. Instead, such a split point also stores the sampled element, and elements whose key equals the split key are placed by the tie-breaker. This keeps buckets near their target size under heavy key collisions, at the price of a comparator call for the colliding elements only.

3.4 Assigning elements

When an element is added, it receives a bucket id for every sort key: a binary search over the split points, $O(\log k)$ integer comparisons for a keyed key and $\lceil \log_2 k \rceil$ comparator calls for a comparator-only key, where $k \leq 64$ is the number of buckets. Ids are stored as 16-bit integers, and for keyed keys the 64-bit normalized key is stored as well, so later comparisons between elements of different normalized keys never call the comparator.

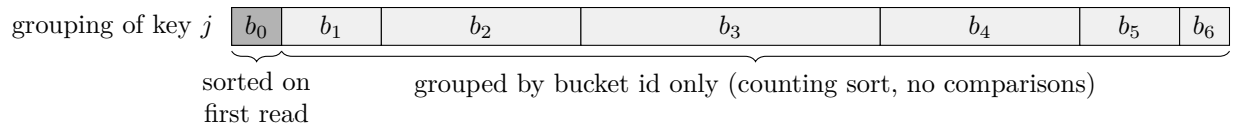


Figure 1: The grouping of one sort key after a column click: positions are ordered by stored bucket id; only the first bucket, about one page, is sorted before the first page is shown. Bucket widths follow the default geometric plan.

3.5 Switching the order

SortBy$(j, \cdot)$ builds the *grouping* of key j: an array of the n element positions, ordered by bucket id, with the start of each bucket. It is a counting sort over the stored ids: one pass to count, a prefix sum over k counters, and one pass to scatter, $O(n + k)$ time and no comparisons. The grouping is cached per key, together with which of its buckets are sorted, so switching back to a column used before costs $O(1)$.

3.6 Lazy sorting

Read$(i, i+p)$ locates the buckets that cover positions i to $i+p-1$ by binary search over the bucket starts and sorts each of them that is not sorted yet, then copies the requested range. Buckets are sorted with a stable merge sort; before sorting, a linear scan checks whether the bucket is already in order, which makes buckets of ties (a column where all values are equal) cost one pass. Showing the first page after a switch therefore sorts about one page of elements. A full order, if needed, is produced by sorting the remaining buckets, for example in an idle callback.

3.7 Descending order and groups

Descending order reads the grouping backwards: bucket $k-1$ first, each bucket reversed. Because the plan is symmetric, the first descending page is as cheap as the first ascending page, and one grouping serves both directions. Ties are the one subtlety: a stable descending sort keeps equal elements in insertion order, so a sorted bucket is reversed run by run, not as a whole.

File managers keep folders before files in both directions. A sort key can therefore have *groups*: a small number of classes (up to 256) that keep ascending order in both directions, while the order inside each group follows the direction. Each group gets its own layout with page-sized end buckets, so the first page of each group, in each direction, is cheap.

3.8 Updates and rebalancing

Additions and updates assign bucket ids as above; removals compact the stored arrays. A layout becomes *stale* when the collection has grown to more than twice the size the layout was estimated from, or when a bucket holds more than four times its expected size. Stale layouts are re-estimated at the end of a bulk addition, or before a read once at least $\max(p, n/8)$ changes have accumulated, which amortizes the re-estimation over the changes that caused it.

4 Analysis

Theorem 1 (Exactness)**.** *For any layout,* Read$(i, i+p)$ *returns the elements at positions* $i, \ldots, i+p-1$ *of a stable sort of* S *by sort key* j *in the chosen direction.*

Table 1: Costs for n elements, k buckets, m sort keys and page size p.

Operation	Cost
add one element	$O(m \log k)$; comparator-only keys: $\lceil \log_2 k \rceil$ comparator calls
layout estimation	sorting a sample of $s \leq 2048$ elements, per key
SortBy	$O(n + k)$, no comparisons; $O(1)$ when the grouping is cached
first page	sorting the bucket(s) that cover it: $O(b \log b)$, $b \approx \max(p, n/s)$
memory	2 B bucket id + 8 B normalized key + 4 B grouping per element and key

Proof. Let $\sigma_0 \prec \cdots \prec \sigma_{k-2}$ be the split points. An element x is in bucket b exactly when $\sigma_{b-1} \preceq x \prec \sigma_b$ (with $\sigma_{-1} = -\infty$, $\sigma_{k-1} = +\infty$); for keyed keys with a stored split element, the comparison against σ_b uses the normalized key and then the tie-breaker, which is the key's order. Hence for x in bucket a and y in bucket b with $a < b$ we have $x \prec y$, and elements that compare equal are in the same bucket. The grouping lists the buckets in increasing id and, by the counting sort, the elements of each bucket in insertion order. Sorting a bucket with a stable sort therefore yields exactly the corresponding segment of the stable sort of S, and the concatenation of all segments is the stable sort itself. A read sorts every bucket it touches before copying, so the positions it returns are final. Descending order reverses the bucket order and, within a bucket, the order of runs of equal elements while keeping each run in insertion order, which is the stable descending sort; with groups the same argument applies within each group. □

The layout thus affects only the cost of reads. A poor sample, a skewed distribution or a plan that does not fit the data can make the first bucket large, which slows the first read, but cannot change the order.

Inconsistent comparators. The theorem assumes a total preorder. Real comparators are not always one (Section 7); with a cyclic comparator no order is correct, and `std::sort` has undefined behaviour. Bucketization only partitions elements and merge-sorts buckets, and neither step can lose or duplicate an element whatever the comparator returns. With an inconsistent comparator, every element is still returned exactly once.

Costs. Table 1 summarizes the costs. For comparator-only keys, loading costs $n\lceil \log_2 k \rceil$ comparator calls per key, against about $n \log_2 n$ for one full sort: with $k = 64$ and $n = 10^5$, about a third. A switch costs $n + k$ memory operations and no comparisons; on 10^6 elements this pass, not the sorting of the first bucket, dominates the switch time.

The size of the first bucket. Split positions are computed in the sample, so a split cannot be placed closer than one sample element to the start: the first bucket holds at least about n/s elements. With $s = 2048$ this is below one page up to about 130,000 elements. Beyond that the first read sorts about n/s elements, still a small fraction of n: on the 1,000,000-entry camera roll the first page cost 113–1,481 element sorts, depending on the column, against 73–241 at 100,000 entries. A larger sample, or a second sampling round inside the first bucket, would lower this bound; we have not needed either.

5 Implementation

The reference implementation is a header-only C++17 library of about 1,800 lines (`speedup::bucketization`). Its interface, `BucketedList<T>`, is implemented by `BucketIndex<T>`:

```
BucketIndex<File> files(BucketPlan::defaults().with_page_size(rows));
files.add_sort_key(SortKey<File>::of("name", &File::name));
files.add_sort_key(SortKey<File>::of("modified", &File::modified)
                        .with_groups(2, [](const File& f) { return f.is_dir ? 0 : 1; }));
files.add_all(list_directory());          // pre-sort: every key, once
files.sort_by("modified", Direction::descending);
for (const File& f : files.range(first, first + rows)) draw(f);
```

A pure-Java port follows the same interface. A cross-language specification fixes the normalized-key encodings, the sampling (a SplitMix64 generator [9] with a fixed seed), the split rule and the bucket ids, so that conforming ports produce identical bucket ids for the same data; both implementations check the same golden values in their test suites. The C++ unit tests also count comparator calls: on 100,000 elements with a comparator-only key, the first page after a switch costs 569 comparisons, against about 1.66 million for a full sort.

The index is not thread-safe, even for concurrent reads, because reads sort lazily; a view model guards it with its existing lock.

6 Evaluation

6.1 Setup

Machines. The reference runs used a 4-vCPU Intel Xeon at 2.1 GHz cloud container with Ubuntu 24.04, GCC 13.3 (`-O3`), Qt 6.4.2, ICU 74.2 and GLib 2.80. The portable benchmarks were repeated on a desktop AMD Ryzen 5 2600X (Zen+, 6 cores, 3.6 GHz base) with 16 GB of DDR4-2133 under Windows 10, with MinGW-w64 GCC 13.2 (`-O3`). Every number is the median of five runs (one run for the 1,000,000-entry dataset in the file-manager stacks), on one thread unless stated otherwise.

Baselines. Each project is measured on its own code path for a column click, extracted from its source at a recorded commit and run on the same data structures it uses: Dolphin's `KFileItemModel` comparison (`QCollator` on ICU, natural ordering, folders first) with its parallel merge sort and the bookkeeping after it; GLib collation keys with `g_sequence_sort` for Thunar, Nemo and Caja and GTK 4's sort model for Nautilus; the Qt sort paths of PCManFM-Qt, the KDE file dialogs and Krusader; nnn's C comparator; the Godot editor's FileSystem dock; and yazi and lf in their native languages (Rust and Go), each through a small harness around the project's sort function. Bucketization uses the same comparison as the project's tie-breaker, so its order is the project's order by construction, and every result is checked element by element against the project's own output. We also compare with `std::partial_sort` of the first page, the obvious "sort only what is visible" alternative.

What is measured. *Re-sort* is the project's time for a column click. *Switch* is Bucketization's SortBy plus the first page (64 rows) without a cached grouping, i.e. the first time the column is used after loading. *Pre-sort* is the one-time cost of computing every element's key and bucket for one column; it can run while the folder is being listed, or on the first click.

Table 2: Synthetic workload, $n = 10^6$, milliseconds. Pre-sort registers all four keys and assigns every element; it is paid once per load. Speed-up is `std::sort` over switch + first page.

Machine	Key	`std::sort`	Pre-sort	Switch + page	Speed-up	Cached switch	Switch + sort all
Xeon (Linux)	name	1,052	135	2.1	501×	0.0045	781
	size	416	130	1.8	235×	0.0049	145
	modified	448	137	2.1	212×	0.0048	151
	accessed	433	119	1.8	238×	0.0050	131
Ryzen (Windows)	name	1,088	152	4.5	243×	0.0038	1,214
	size	363	138	4.3	85×	0.0048	135
	modified	419	137	4.0	104×	0.0050	147
	accessed	358	139	4.5	79×	0.0047	128

Data. Real directory listings: `/usr/bin` (1,172 entries), `/usr/lib/x86_64-linux-gnu` (1,533), the top level of a web application's `node_modules` (395) and the same tree flattened (36,069), all of `/usr` flattened (97,974), and the Linux kernel source tree (97,005, all timestamps equal). Two synthetic camera rolls (100,000 and 1,000,000 `IMG_...` names) and a 20,000-name multilingual set used for correctness checks complete the data.

6.2 Synthetic workload

Table 2 shows the micro-benchmark: one million synthetic files (a third camera names with long shared prefixes, a third numbered documents, a third random words; log-normal sizes; random timestamps), four columns, compared with one `std::sort` of the column. Showing the first page after a switch is two to three orders of magnitude faster than a full sort, a cached switch costs microseconds, and pre-sorting all four columns costs less than one full sort of any of them. Sorting everything after the switch costs about a third of a full sort for the numeric columns, whose buckets are already in order relative to each other. For names it is not a saving: on the Xeon it took 781 ms against 1,052 ms for `std::sort`, and on the Ryzen 1,214 ms against 1,088 ms. Camera names share long prefixes, so their normalized keys collide and most comparisons inside the buckets fall through to the full string comparison, in a stable merge sort. Sorting everything is not what Bucketization is for, but a view that needs the full order of a name column at once gains nothing from it.

6.3 Real file managers

Table 3 summarizes the twelve projects on the large real listings and the 100,000-photo camera roll (four datasets, three columns, two directions). In every project and every case, the visible rows are final within 0.047–3.0 ms after the click, against 4.3–3,050 ms for the project's own re-sort. The largest gains are where comparisons are expensive: the Qt-based projects compare names with `QCollator` on every comparison, and lf normalizes both names on every comparison. The smallest are where the project already sorts fast: yazi's Rust comparator allocates nothing, and the GNOME family compares precomputed collation keys with `strcmp`.

Against partial sorting. `std::partial_sort` of the first page with Dolphin's comparison is the strongest simple alternative. On the real listings it is a median 9.1× slower than a Bucketization switch (2.9–849×), and it repeats its work on every click. Its worst case is a column of equal keys: on the kernel tree sorted by date, where all timestamps are equal, every comparison falls through to Dolphin's name comparison and the partial sort takes 212 ms, against 0.25 ms for Bucketization, whose layout splits the run of equal keys with the tie-breaker.

Table 3: Column clicks on the large listings (36,069–100,000 entries), Xeon reference machine, milliseconds; ranges over datasets, columns and directions. *First click slower* counts cases where pre-sort + switch exceeded the project's re-sort. *Valid orders* counts every dataset, column and direction, including the small folders; for Dolphin it includes the cases where Dolphin's own comparator has cycles and no exact order exists (Section 7).

Project	Re-sort today (ms)	Switch + page (ms)	Speed-up	Pre-sort, once (ms)	First click slower	Valid orders
Dolphin	59–713	0.080–0.72	178–3,842×	2.1–73	0/24	54/54
Thunar	38–333	0.047–0.30	218–1,552×	2.8–10	0/24	42/42
Nemo	36–382	0.057–0.22	380–2,206×	2.6–11	0/24	42/42
Caja	45–389	0.048–0.24	312–1,953×	2.5–10	0/24	42/42
Nautilus	63–480	0.052–0.26	729–2,921×	2.6–12	0/24	42/42
PCManFM-Qt	48–1,499	0.092–2.0	322–7,640×	2.6–206	0/24	42/42
KIO dialogs	84–1,384	0.079–1.2	356–6,647×	2.0–159	0/24	42/42
Krusader	115–801	0.056–0.54	587–4,207×	1.9–68	0/24	42/42
nnn	7.7–290	0.072–0.28	72–1,379×	1.9–106	0/24	42/42
yazi (native Rust)	6.3–63	0.060–0.33	37–425×	1.5–12	0/24	42/42
lf (native Go)	8.0–3,050	0.057–1.0	8.3–12,108×	1.3–184	0/24	42/42
Godot	4.3–1,165	0.089–3.0	13–871×	1.2–1,261	6/24	36/36

Opening a folder. Dolphin sorts all entries before showing the first row of a folder. Bucketization sorts the visible page after pre-sorting the visible column: on the three large real listings 13–40 ms against 208–434 ms (11–16×), and 33–92 ms if all three columns are pre-sorted eagerly (4.7–6.4×). Opening therefore becomes faster too, provided the other columns are pre-sorted lazily, on first use, or in the background.

Comparator-only keys. Without a normalized key (Dolphin's comparison only), switching is as fast as in keyed mode, because a switch compares nothing either way, but the pre-sort costs 0.7–3.2× the keyed pre-sort on the large real listings: every assignment calls the comparator $\lceil \log_2 k \rceil$ times, while a keyed assignment mostly compares integers.

6.4 Where it does not help

Small folders. Under about 2,000 entries every project re-sorts in a few milliseconds, below one 60 Hz frame (16.7 ms). The gain is real but invisible. Pre-sorting all three columns of the 395-entry folder eagerly is the one case where Bucketization measured slower than Dolphin (1.39 against 1.03 ms). An integration should keep the existing sort for small collections and pre-sort columns on first use.

The first click. The pre-sort is the price of the first click on a column, a median of 7.0 ms on the large listings. It rises to about 200 ms where the 64-bit key cannot tell the elements apart and the project's comparator has to break the ties: camera names under ICU collation keys, identical timestamps tie-broken by name through `QCollator`, and nnn's names, whose key needs `strxfrm` per file. It stays below one full re-sort of the same folder in every file manager. The exception is Godot's FileSystem dock, whose own sort is fast for some columns: there the first click (pre-sort + switch) was slower than Godot's re-sort in 6 of 24 large cases; later clicks are not.

6.5 Second machine

Table 4 repeats the projects that build without Linux desktop libraries on the Ryzen desktop under Windows: nnn, the C++ port of lf, Godot, and yazi natively in Rust (lf's Go program needs a

Table 4: The portable projects on the Ryzen 5 2600X under Windows 10 (MinGW-w64 GCC 13.2), milliseconds, same datasets and method as Table 3. Timings cover only the orders that passed the order check; for nnn, its name column failed it on Windows (Section 6.5).

Project	Re-sort today (ms)	Switch + page (ms)	Speed-up	Pre-sort, once (ms)	First click slower	Valid orders
nnn	36–2,257	0.14–0.69	145–4,639×	2.6–285	0/18	32/42
yazi (native Rust)	7.7–67	0.089–0.43	40–286×	1.3–16	0/24	42/42
lf (C++ port)	4.0–1,904	0.080–1.2	8.8–6,503×	1.1–336	0/24	42/42
Godot	3.6–2,391	0.13–6.4	5.2–879×	1.6–2,510	8/24	36/36

newer Go than was installed). Table 2 includes the synthetic workload on that machine. The conclusions carry over. The switch takes 0.080–6.4 ms, against 3.6–2,391 ms for the projects' own re-sorts (median speed-up 148×). nnn's own re-sort is much slower on Windows than on Linux; it compares names with the C library's `strcoll`, whose cost differs between the two runtimes. Godot's first click was slower than its own re-sort in 8 of 24 large cases, against 6 on Linux. The switch itself is about twice as slow as on the Xeon (Table 2). It is mostly a pass over memory, and the desktop's memory ran at only 2133 MT/s, which is a likely cause.

One column could not be reproduced on Windows. nnn's name column failed the order check in 10 of 42 cases on Windows, and in none on Linux. Bucketization's key for it is the first eight bytes of `strxfrm` output, which the C standard requires to order strings like `strcoll`. The Windows C runtime (UCRT, used by both MSVC and MinGW-w64) breaks that rule in its English locales: `strcoll` places `x-terminal-emulator` before `xterm`, and `strcmp` on the `strxfrm` output after it. Our checker (`speedup_check_strxfrm`) finds 245 of 686,206 pairs of names in `/usr/bin` and 305 of 1,174,278 in `/usr/lib` ordered differently in `en_US.UTF-8`, and none in the `C` locale. A key built from `strxfrm` is therefore not order-preserving on Windows and cannot reproduce `strcoll`'s order there; a comparator-only sort key, which calls `strcoll` itself, can. Table 4 reports nnn's timings for the orders that passed the check only.

7 Comparator defects found

Checking Bucketization's output against each project's own order requires that order to be well defined. Three projects' comparators turned out not to be:

Dolphin's natural comparison has cycles. It reads a two-part number such as `6.10` as a decimal ($6.10 < 6.9$) but compares two-part with three-part numbers part by part ($6.9.1 < 6.10$). Ordinary file names therefore form cycles: `kernel-6.10.tar.xz` < `kernel-6.9.tar.xz` < `kernel-6.9.1.tar.xz` < `kernel-6.10.tar.xz`. No sort can produce a consistent order for such names; the order Dolphin shows depends on the order in which the folder was listed. On the multilingual set, even `std::stable_sort` with Dolphin's comparison disagrees with Dolphin's own merge sort in 32 of 20,000 positions.

Krusader's descending comparator is not a strict weak ordering. It is defined as the negation of the ascending one, so for two items that compare equal (`README` and `readme` under the default case-insensitive setting) it returns true in both directions, and it returns true for an item compared with itself. It is passed to `std::stable_sort` and `std::lower_bound`, whose behaviour is then undefined; libstdc++'s debug mode reports the violation.

lf normalizes names on every comparison. Its natural ordering removes diacritics and lowers the case of both names inside the comparator, so each sort allocates and normalizes $O(n \log n)$ strings. Normalizing once per file makes name sorting 4–8 times faster on large folders. This is a performance issue, not a correctness one.

All three were reported to the projects in October 2026, independently of Bucketization. Because comparators like these exist, Bucketization sorts buckets only with merge sorts, which tolerate inconsistent comparators (Section 4).

8 Related work

Distribution sorting. Assigning elements to buckets delimited by sampled splitters is samplesort [3]; super scalar samplesort [8] and its in-place parallel successor [1] make the assignment branch-free over an implicit search tree of splitters. Bucketization uses the same assignment, but stops after it, keeps the bucket ids of every key, and sorts buckets only on demand. Its buckets have planned, unequal sizes, small at both ends of the order, while samplesort aims for equal buckets.

Partial and incremental sorting. Partial quicksort [6] produces the p smallest elements in order in $O(n + p \log p)$ expected time, and incremental sorting [7] produces the elements in order on demand. Both compare all n elements per query and keep no state across different sort keys; Bucketization performs these comparisons at load time, once per key, and a switch compares nothing.

Adaptive indexing. Database cracking [5] reorganizes a column incrementally as a side effect of the range queries that touch it, so that the work follows the workload. Lazy bucket sorting follows the same principle, with the view's scroll position as the workload.

Normalized and abbreviated keys. Database systems compare order-preserving binary prefixes of keys and fall back to full comparisons only on ties [4]; PostgreSQL's abbreviated keys apply this to collated text [10]. Bucketization's normalized keys are such prefixes, used both for bucket assignment and inside bucket sorts.

Per-column sorted indexes. A view could also sort every column fully when the data is loaded and keep the permutations, making every later switch $O(1)$. That costs m full sorts at load ($\Theta(mn \log n)$ comparisons) and $O(\log n)$ comparator calls per update and key; Bucketization needs $mn\lceil \log_2 k \rceil$ comparisons at load and sorts only what is shown, which is what makes opening a folder faster rather than slower.

9 Limitations

The file-manager measurements cover the sorting stage only, not listing the directory, creating items or drawing the view, and the extracted sort paths were run outside the applications. The reference results come from one cloud container; the second machine covers only the projects that build without Linux desktop libraries. Dolphin's name sort uses four threads in our measurements while Bucketization uses one, which favours Dolphin. The camera rolls and the multilingual set are synthetic. Bucketization costs 14 bytes per element and sort key, plus one pointer or handle per element. Finally, the design targets views that are read from the start or the end; a view that jumps to the middle of a large collection sorts one of the large middle buckets on the first read.

10 Conclusion

Re-sorting a multi-column view on every click spends almost all of its time on elements the user cannot see. Bucketization assigns every element to a bucket of every column once, when it is loaded, and turns the click into a counting pass and the sorting of one page-sized bucket, while keeping the order exact. In the sort code of twelve real projects this moved column switches on large folders from tens or hundreds of milliseconds, sometimes seconds, to well under a frame. The C++ and Java implementations, all benchmarks, the captured listings and the scripts that produced every number in this paper are open source.

Availability

Source code, data and scripts: `https://github.com/attilatorda/SpeedUp` (AGPL-3.0, with an additional permission for the file managers named in the repository). The numbers in this paper are generated from the result files in `benchmarks/outputs/` by `benchmarks/scripts/bucketization_paper_numbers.py`.

References

[1] Michael Axtmann, Sascha Witt, Daniel Ferizovic, and Peter Sanders. In-place parallel super scalar samplesort (IPS^{4}o). In *25th Annual European Symposium on Algorithms (ESA 2017)*, volume 87 of *LIPIcs*, pages 9:1–9:14, 2017. doi: 10.4230/LIPIcs.ESA.2017.9.

[2] Mark Davis, Ken Whistler, and Markus Scherer. Unicode technical standard #10: Unicode collation algorithm. `https://www.unicode.org/reports/tr10/`. The Unicode Consortium.

[3] W. D. Frazer and A. C. McKellar. Samplesort: A sampling approach to minimal storage tree sorting. *Journal of the ACM*, 17(3):496–507, 1970. doi: 10.1145/321592.321600.

[4] Goetz Graefe. Implementing sorting in database systems. *ACM Computing Surveys*, 38(3): Article 10, 2006. doi: 10.1145/1132960.1132964.

[5] Stratos Idreos, Martin L. Kersten, and Stefan Manegold. Database cracking. In *Third Biennial Conference on Innovative Data Systems Research (CIDR)*, pages 68–78, 2007.

[6] Conrado Martínez. Partial quicksort. In *Proceedings of the 6th ACM-SIAM Workshop on Algorithm Engineering and Experiments and the 1st ACM-SIAM Workshop on Analytic Algorithmics and Combinatorics (ALENEX/ANALCO)*, pages 224–228. SIAM, 2004.

[7] Rodrigo Paredes and Gonzalo Navarro. Optimal incremental sorting. In *Proceedings of the 8th Workshop on Algorithm Engineering and Experiments (ALENEX)*, pages 171–182. SIAM, 2006.

[8] Peter Sanders and Sebastian Winkel. Super scalar sample sort. In *Algorithms – ESA 2004*, volume 3221 of *Lecture Notes in Computer Science*, pages 784–796. Springer, 2004.

[9] Guy L. Steele, Jr., Doug Lea, and Christine H. Flood. Fast splittable pseudorandom number generators. In *Proceedings of the 2014 ACM International Conference on Object Oriented Programming Systems Languages & Applications (OOPSLA)*, pages 453–472, 2014. doi: 10.1145/2660193.2660195.

[10] The PostgreSQL Global Development Group. PostgreSQL 9.5 release notes: abbreviated keys for sorting `varchar`, `text` and `numeric`. `https://www.postgresql.org/docs/release/9.5.0/`, 2016.